

**SIMATS ENGINEERING**

**ASSESSMENT TOOL 1**

**COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS**

**COURSE CODE: CSA06**

**COURSE OUTCOME COVERED**

**CO 1:** Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

(BL-1, BL-2, BL-3)

**Assessment Tool Weightage:** 40%

**QUESTIONS:** 10

Annexure A

**Application-Based Question**

**Total Marks: 50**

***Code of Conduct:***

*I*

**S.PRITHISH (192421134)** (Name/Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that **any** violation of academic integrity rules will result in disciplinary **action**.

***Signature of the student:***

**S.PRITHISH**

Annexure A

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import time

# -----
# Page Configuration
# -----
st.set_page_config(page_title="Bank Fraud Monitoring", layout="wide")

st.title("🏦 Bank Transaction Fraud Monitoring")
st.write("### Demonstration of Three Nested Loops")

# -----
# Sidebar
# -----
st.sidebar.header("Simulation Settings")

customers = st.sidebar.slider("Customers", 2, 20, 5)
transactions = st.sidebar.slider("Transactions per Customer", 2, 20, 5)
checks = st.sidebar.slider("Security Checks", 2, 20, 5)

# -----
# Run Button
# -----
if st.button("Run Monitoring"):

    st.subheader("Customer Database")

    customer_data = []

    for i in range(customers):
        customer_data.append({
```

```

        "Customer ID": f"CUST-{1001+i}",
        "Account Balance": np.random.randint(5000, 100000),
        "Account Type": np.random.choice(["Savings", "Current"])
    })

df = pd.DataFrame(customer_data)

st.dataframe(df, use_container_width=True)

progress = st.progress(0)

report = []

operations = 0

fraud_count = 0

start = time.time()

# -----
# Three Nested Loops
# -----
for i in range(customers):

    customer = df.iloc[i]

    suspicious = False

    for j in range(transactions):

        amount = np.random.randint(100, 100000)

        for k in range(checks):

            score = np.random.randint(1, 100)

            operations += 1

            if amount > 80000 and score > 70:
                suspicious = True

```

```

        if suspicious:
            fraud_count += 1

    report.append({
        "Customer": customer["Customer ID"],
        "Balance": customer["Account Balance"],
        "Result": "⚠️ Fraud Alert" if suspicious else "✅ Safe"
    })

    progress.progress((i + 1) / customers)

end = time.time()

result = pd.DataFrame(report)

st.subheader("Fraud Detection Result")

st.dataframe(result, use_container_width=True)

# -----
# Metrics
# -----
st.subheader("Performance Analysis")

col1, col2, col3, col4 = st.columns(4)

col1.metric("Operations", operations)
col2.metric("Fraud Alerts", fraud_count)
col3.metric("Execution Time", f"{end-start:.6f} sec")
col4.metric("Space Used", customers + transactions + checks)

st.success("Fraud Monitoring Completed Successfully!")

st.divider()

# -----
# Graph
# -----
st.subheader("Operations Growth")

```

```

x = []
y = []

for n in range(1, customers + 1):
    x.append(n)
    y.append(n * transactions * checks)

graph = pd.DataFrame({
    "Customers": x,
    "Operations": y
})

fig = px.bar(
    graph,
    x="Customers",
    y="Operations",
    color="Operations",
    title="Growth of Three Nested Loops"
)

st.plotly_chart(fig, use_container_width=True)

st.divider()

# -----
# Pie Chart
# -----

safe = customers - fraud_count

pie = pd.DataFrame({
    "Status": ["Safe", "Fraud"],
    "Count": [safe, fraud_count]
})

fig2 = px.pie(
    pie,
    values="Count",
    names="Status",
    title="Fraud Distribution"
)

```

```

)

st.plotly_chart(fig2, use_container_width=True)

st.divider()

# -----
# Complexity
# -----
st.subheader("Algorithm Complexity")

st.code("""
for each Customer:
    for each Transaction:
        for each Security Check:
            Analyze Transaction
""")

st.markdown("""
### Time Complexity

** $O(\text{Customers} \times \text{Transactions} \times \text{Security Checks})$ **

The algorithm visits every customer.

For every customer it checks every transaction.

For every transaction it performs all security checks.

Therefore,

**Time Complexity =  $O(n^3)$ ** (when all three inputs grow together)

---

### Space Complexity

The program stores only customer information and final results.

**Space Complexity =  $O(n)$ **

```

```
---
```

```
### Total Operations
```

```
Operations = Customers × Transactions × Security Checks
```

```
""")
```

## REPORT

### SCREENSHOTS OF SOURCE CODE

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import time

# -----
# Page Configuration
# -----
st.set_page_config(page_title="Bank Fraud Monitoring", layout="wide")

st.title("🏦 Bank Transaction Fraud Monitoring")
st.write("### Demonstration of Three Nested Loops")

# -----
# Sidebar
# -----
st.sidebar.header("Simulation Settings")

customers = st.sidebar.slider("Customers", 2, 20, 5)
transactions = st.sidebar.slider("Transactions per Customer", 2, 20, 5)
checks = st.sidebar.slider("Security Checks", 2, 20, 5)

# -----
# Run Button
# -----
if st.button("Run Monitoring"):

    st.subheader("Customer Database")

    customer_data = []

    for i in range(customers):
        customer_data.append({
```



```

        "Customer ID": f"CUST-{1001+i}",
        "Account Balance": np.random.randint(5000, 100000),
        "Account Type": np.random.choice(["Savings", "Current"])
    })

df = pd.DataFrame(customer_data)

st.dataframe(df, use_container_width=True)

progress = st.progress(0)

report = []

operations = 0

fraud_count = 0

start = time.time()

# -----
# Three Nested Loops
# -----
for i in range(customers):

    customer = df.iloc[i]

    suspicious = False

    for j in range(transactions):

        amount = np.random.randint(100, 100000)

        for k in range(checks):

            score = np.random.randint(1, 100)

            operations += 1

            if amount > 80000 and score > 70:
                suspicious = True

```

```

        if suspicious:
            fraud_count += 1

    report.append({
        "Customer": customer["Customer ID"],
        "Balance": customer["Account Balance"],
        "Result": "⚠️ Fraud Alert" if suspicious else "✅ Safe"
    })

    progress.progress((i + 1) / customers)

end = time.time()

result = pd.DataFrame(report)

st.subheader("Fraud Detection Result")

st.dataframe(result, use_container_width=True)

# -----
# Metrics
# -----
st.subheader("Performance Analysis")

col1, col2, col3, col4 = st.columns(4)

col1.metric("Operations", operations)
col2.metric("Fraud Alerts", fraud_count)
col3.metric("Execution Time", f"{end-start:.6f} sec")
col4.metric("Space Used", customers + transactions + checks)

st.success("Fraud Monitoring Completed Successfully!")

st.divider()

# -----
# Graph
# -----
st.subheader("Operations Growth")

```

```

x = []
y = []

for n in range(1, customers + 1):
    x.append(n)
    y.append(n * transactions * checks)

graph = pd.DataFrame({
    "Customers": x,
    "Operations": y
})

fig = px.bar(
    graph,
    x="Customers",
    y="Operations",
    color="Operations",
    title="Growth of Three Nested Loops"
)

st.plotly_chart(fig, use_container_width=True)

st.divider()

# -----
# Pie Chart
# -----

safe = customers - fraud_count

pie = pd.DataFrame({
    "Status": ["Safe", "Fraud"],
    "Count": [safe, fraud_count]
})

fig2 = px.pie(
    pie,
    values="Count",
    names="Status",
    title="Fraud Distribution"
)

```

```

)

st.plotly_chart(fig2, use_container_width=True)

st.divider()

# -----
# Complexity
# -----
st.subheader("Algorithm Complexity")

st.code("""
for each Customer:
    for each Transaction:
        for each Security Check:
            Analyze Transaction
""")

st.markdown("""
### Time Complexity

** $O(\text{Customers} \times \text{Transactions} \times \text{Security Checks})$ **

The algorithm visits every customer.

For every customer it checks every transaction.

For every transaction it performs all security checks.

Therefore,

**Time Complexity =  $O(n^3)$ ** (when all three inputs grow together)

---

### Space Complexity

The program stores only customer information and final results.

**Space Complexity =  $O(n)$ **

```

```
---

### Total Operations

Operations = Customers × Transactions × Security Checks

""" )
```

OUTPUT

### Simulation Settings

Customers

5

Transactions per Customer

5

Security Checks

5



## Bank Transaction Fraud Monitoring

### Demonstration of Three Nested Loops

Run Monitoring

|   | Customer  | Balance | Result        |
|---|-----------|---------|---------------|
| 0 | CUST-1001 | 78819   | ✔ Safe        |
| 1 | CUST-1002 | 28895   | ⚠ Fraud Alert |
| 2 | CUST-1003 | 92086   | ⚠ Fraud Alert |
| 3 | CUST-1004 | 54950   | ✔ Safe        |
| 4 | CUST-1005 | 31494   | ✔ Safe        |

Performance Analysis

|            |              |                |            |
|------------|--------------|----------------|------------|
| Operations | Fraud Alerts | Execution Time | Space Used |
| 125        | 2            | 0.004003 ...   | 15         |

Fraud Monitoring Completed Successfully!

ANALYSIS

